

Chap 05. 합성곱 신경망 I

발표자 : 임 정 은
일 시 : 2026.04.26

CONTENTS

5.1 합성곱 신경망

5.1.1 합성곱층의 필요성

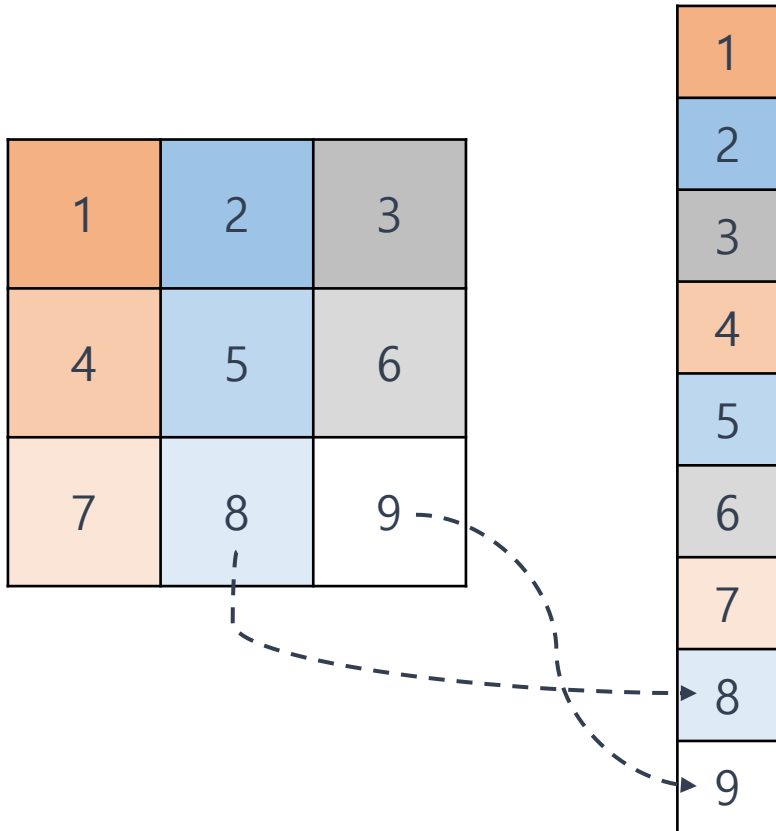
5.1.2 합성곱 신경망 구조

5.1.3 1D, 2D, 3D 합성곱

5.1.1 합성곱층의 필요성

- 합성곱 신경망이란?

음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망으로,
이미지 전체를 한 번에 계산하는 것이 아닌 이미지의 국소적 부분을 계산함으로써 시간과
자원을 절약하여 이미지의 세밀한 부분까지 해석할 수 있는 신경망

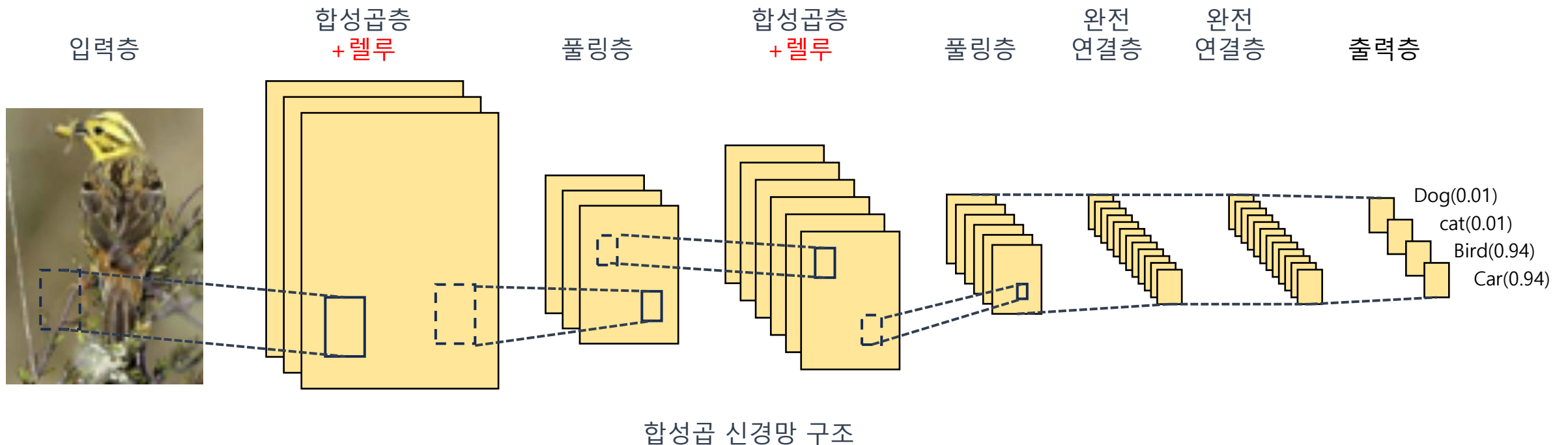


이미지 데이터를 펼쳐서 분석하면 데이터의 공간적 구조가 무시됨.
데이터의 공간적 구조를 고려하기 위해 합성곱층 도입

5.1.2 합성곱 신경망 구조

- 합성곱 신경망(Convolutional Neural Network, CNN, ConvNet)

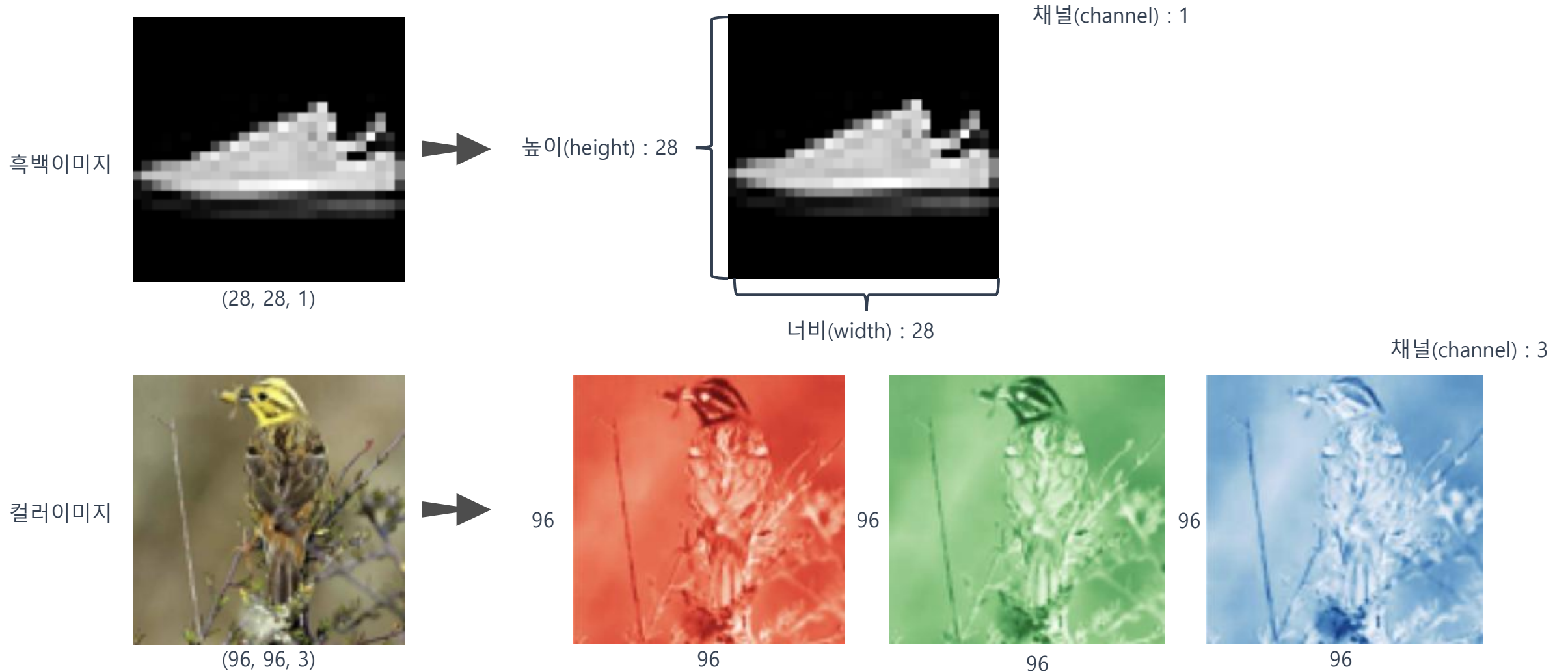
다차원 배열 데이터를 처리하도록 구성되어, 컬러 이미지 같은 다차원 배열 처리에 특화
5개의 층으로 구성(입력층, 합성곱층, 풀링층, 완전연결층, 출력층)



5.1.2 합성곱 신경망 구조

1) 입력층(Input layer)

입력 이미지 데이터가 최초로 거치게 되는 계층



5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

입력 데이터에서 **특성을 추출**하는 역할을 수행

→ 이미지에 대한 특성을 감지하기 위해 **커널(kernel)**이나 **필터(filter)**를 사용

→ 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출하게 되는데,
추출된 결과물을 **특성 맵(feature map)**이라 한다.

→ 필터를 적용하는 위치 간격인 **스트라이드(stride)**에 따라 순차적으로 이동

5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

1단계. 입력 이미지에 3x3 필터 적용

이미지 크기 (6,6,1), 커널/필터 크기 3x3, 스트라이드 1

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$



5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

2단계. 필터가 1만큼 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$



5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

3단계. 필터가 1만큼 두번째 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$



5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

4단계. 필터가 1만큼 세번째 이동

입력

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

필터/커널

1	0	1
0	1	0
1	0	1

출력

3	1	1	3

5단계. 필터가 1만큼 네번째 이동

입력

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

필터/커널

1	0	1
0	1	0
1	0	1

출력

3	1	1	3

6단계. 필터가 1만큼 마지막으로 이동

입력

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

필터/커널

1	0	1
0	1	0
1	0	1

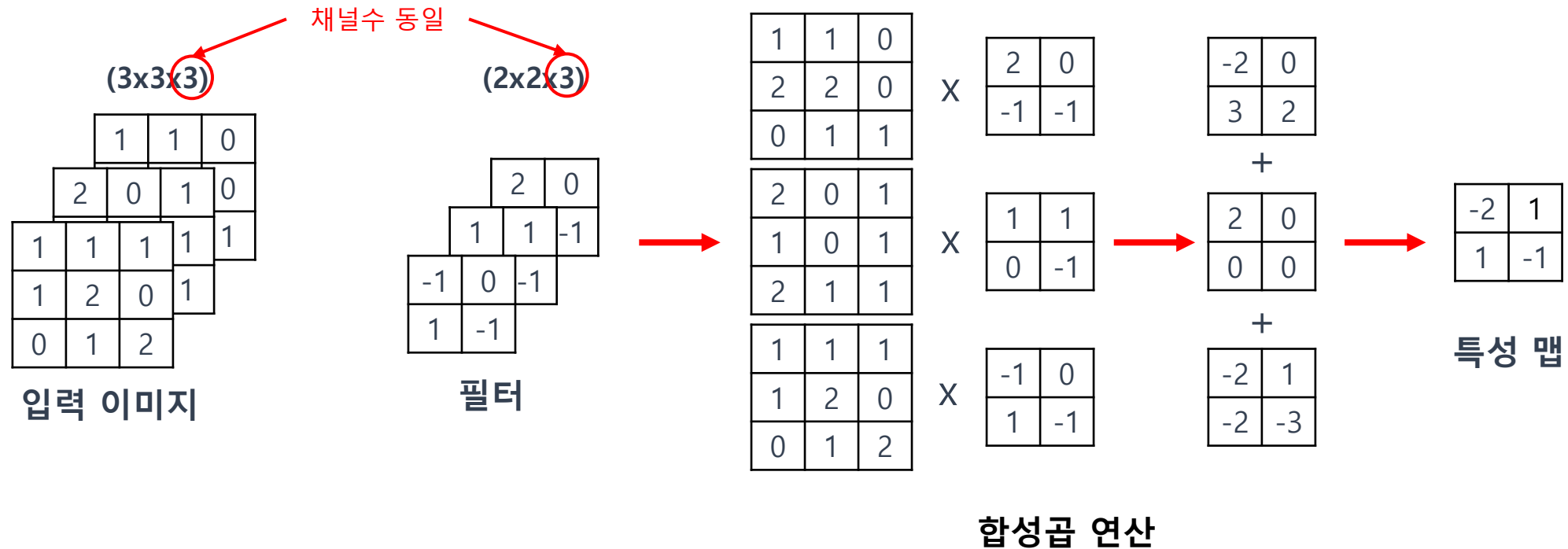
출력

3	1	1	3
1	2	3	0
1	2	2	2
3	0	3	1

5.1.2 합성곱 신경망 구조

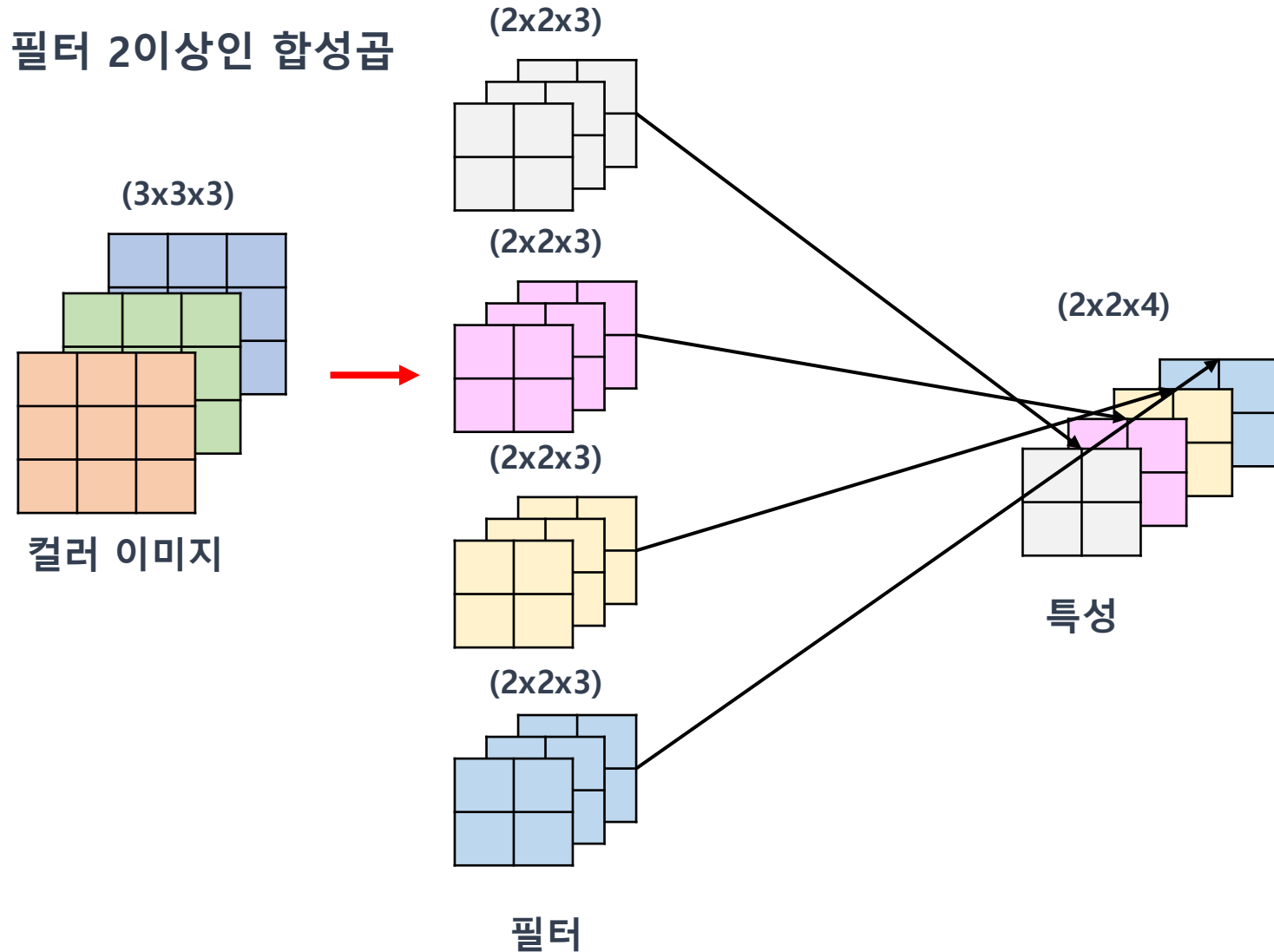
2) 합성곱층(convolutional layer)

컬러 이미지의 합성곱(필터 1개)



5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)



- 입력 데이터 : $W_1 \times H_1 \times D_1$
(W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
- 하이퍼 파라미터
 - 필터개수 : K
 - 필터크기 : F
 - 스트라이드 : S
 - 패딩 : P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$
 - $D_2 = K$

5.1.2 합성곱 신경망 구조

2) 합성곱층(convolutional layer)

- 패딩(padding) : 입력 데이터 주위를 가상의 원소로 채우는 것

1	1	1
1	2	0
0	1	2

입력 이미지



0	0	0	0	0
0	1	1	1	0
0	1	2	0	0
0	0	1	2	0
0	0	0	0	0

→ 이미지 주변 정보를 잃어버리지 않게 해준다.

- 팽창(dilation) : 필터 내부 간격을 확장시켜, 더 넓은 영역의 특징을 추출하는 방법

dilation : 2

1	0	1
0	1	0
1	0	1

필터



1	0	0	0	1
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
1	0	0	0	1

→ 적은 비용으로 receptive field 를 늘린다.

- 출력 데이터 (D : dilation)

$$W_2 = (W_1 - D(F - 1) - 1 + 2P)/S + 1$$

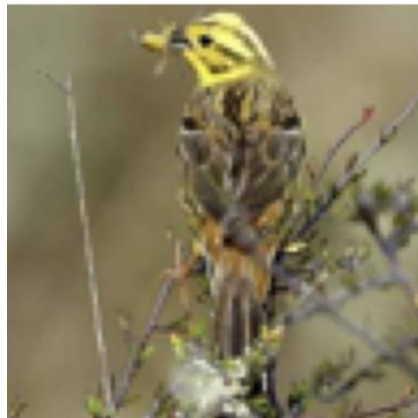
$$H_2 = (H_1 - D(F - 1) - 1 + 2P)/S + 1$$

$$D_2 = K$$

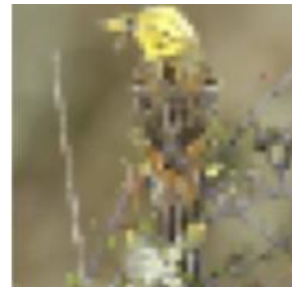
5.1.2 합성곱 신경망 구조

3) 풀링층(pooling layer)

특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함.



→
다운 샘플링



- 최대 풀링(max pooling) : 대상 영역에서 **최댓값**을 추출
- 평균 풀링(average pooling) : 대상 영역에서 **평균**을 반환

5.1.2 합성곱 신경망 구조

3) 풀링층(pooling layer)

최대 풀링 과정



5.1.2 합성곱 신경망 구조

3) 풀링층(pooling layer)

평균 풀링 과정



$$0 = (3 + (-1) + (-3) + 1)/4$$



$$3 = (12 + (-1) + 0 + 1)/4$$



$$0 = (2 + (-3) + 3 + (-2))/4$$

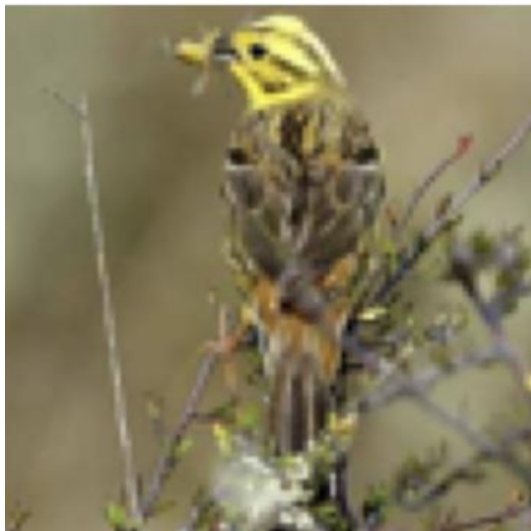


$$1 = (0 + 1 + 4 + (-1))/4$$

5.1.2 합성곱 신경망 구조

3) 풀링층(pooling layer)

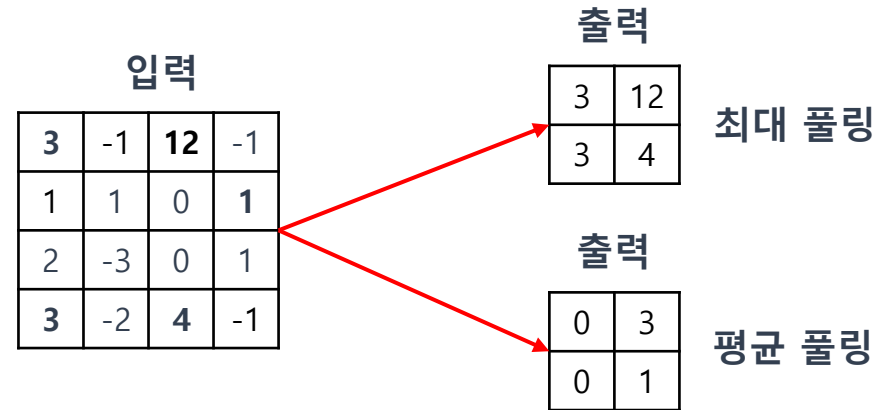
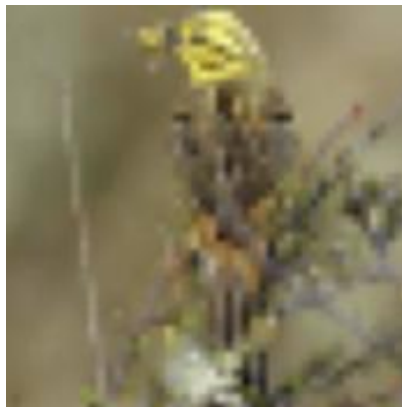
최대 풀링과 평균 풀링 비교



최대풀링
→



평균풀링
→

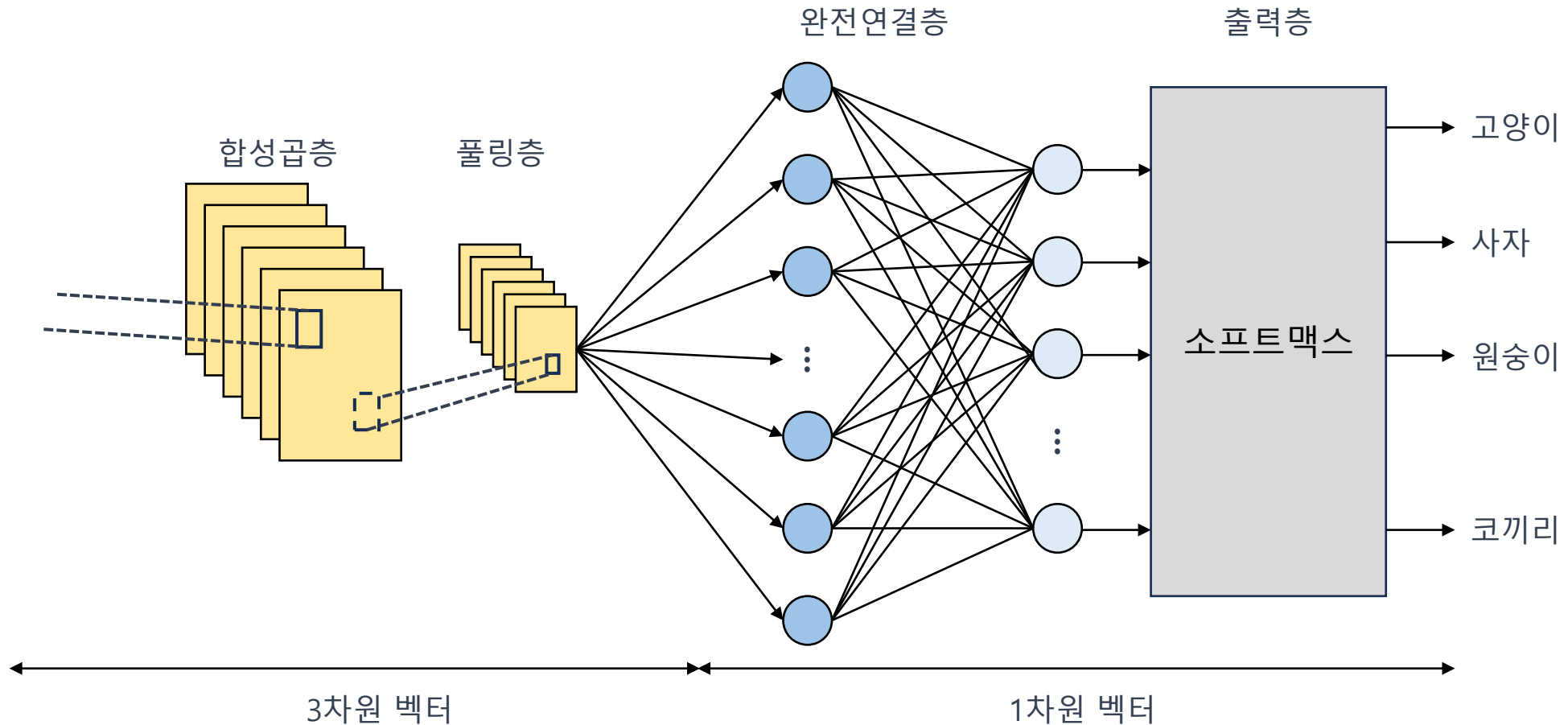


- 입력 데이터 : $W_1 \times H_1 \times D_1$
(W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
- 하이퍼 파라미터
필터크기 : F
스트라이드 : S
패딩 : P
- 출력 데이터
 $W_2 = (W_1 - F)/S + 1$
 $H_2 = (H_1 - F)/S + 1$
 $D_2 = D_1$

5.1.2 합성곱 신경망 구조

4) 완전연결층(fully connected layer)

합성곱층과 풀링층을 거치면서 공간적 크기가 줄어든 특성 맵은 최종적으로 완전연결층으로 전달되어 3차원 벡터에서 1차원 벡터로 펼쳐지게(**flatten**) 됨.



5.1.2 합성곱 신경망 구조

5) 출력층(output layer)

출력층에서는 소프트맥스 활성화 함수를 사용

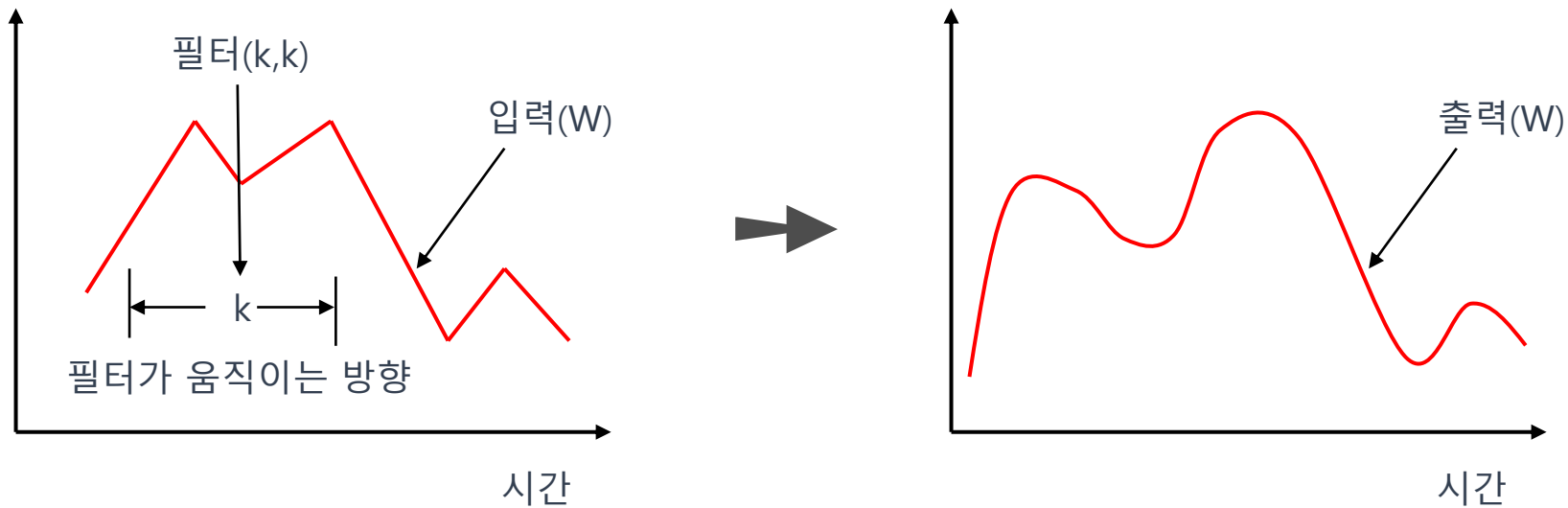
각 레이블에 속할 확률이 가장 높은 값을 갖는 레이블이 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

1) 1D 합성곱

1D 합성곱은 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱

입력(W)과 필터(k)에 대한 출력은 W 로 그래프 곡선을 완화할 때 많이 사용



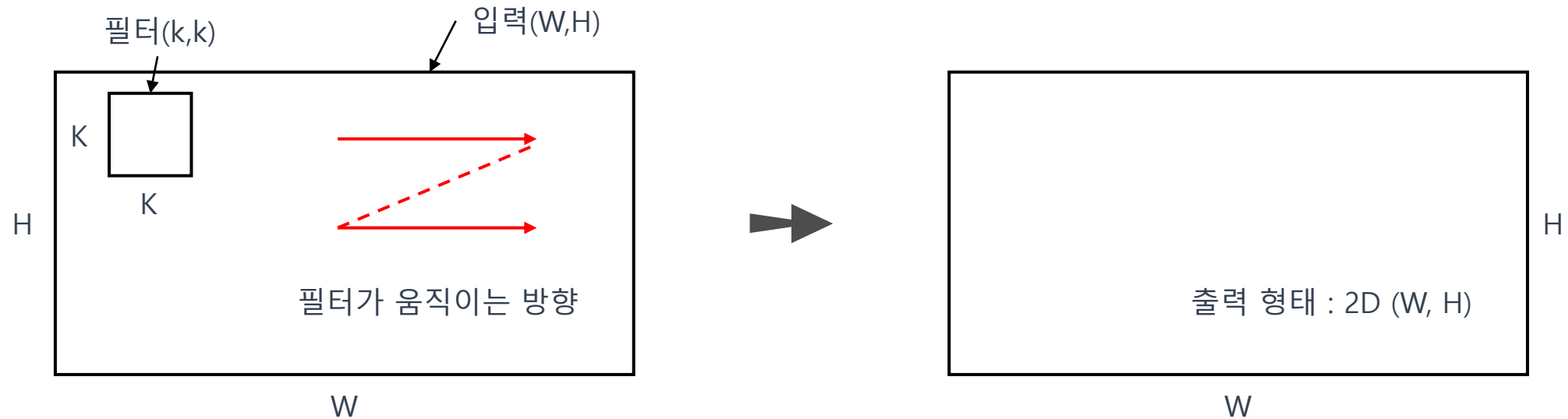
- 입력 : W 너비(Width)
- 필터 : k (너비)
- 출력 : W 너비(Width)

5.1.3 1D, 2D, 3D 합성곱

2) 2D 합성곱

2D 합성곱은 필터가 방향 두개로 움직이는 형태

입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)



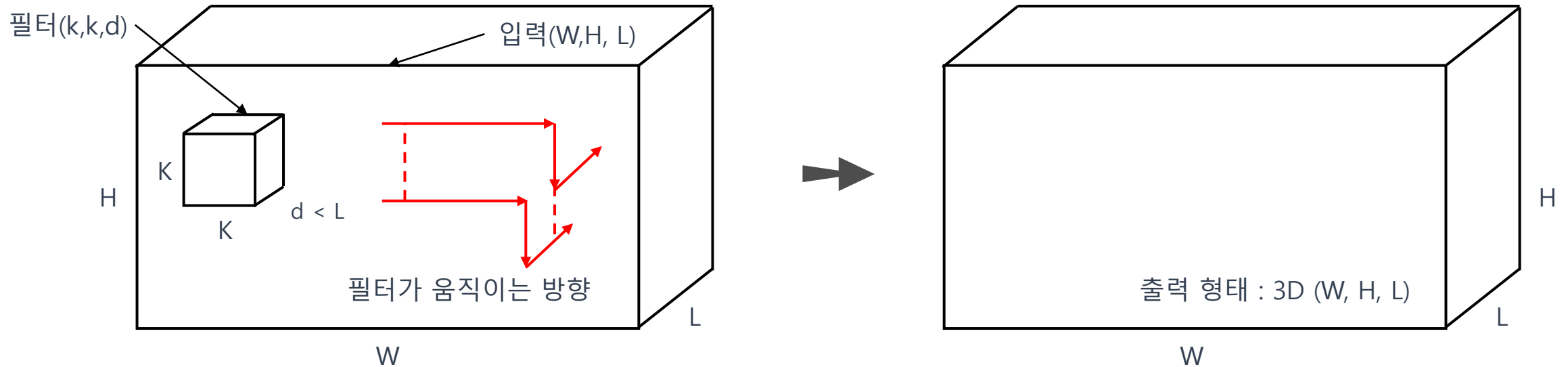
- 입력 : W 너비(Width), H 높이(Height)
- 필터 : $k \times k$ (높이 $\times$ 너비)
- 출력 : W 너비(Width), H 높이(Height)

5.1.3 1D, 2D, 3D 합성곱

3) 3D 합성곱

3D 합성곱은 필터가 세 방향으로 움직이는 형태

입력(W, H, L)과 필터(k, k, d)에 대한 출력은 (W, H, L), 이때 $d < L$



- 입력 : W 너비(Width), H 높이(Height), L 길이(Length)
- 필터 : $k \times k$ (높이 $\times$ 너비), d 깊이(depth)
- 출력 : W 너비(Width), H 높이(Height), L 길이(Length)

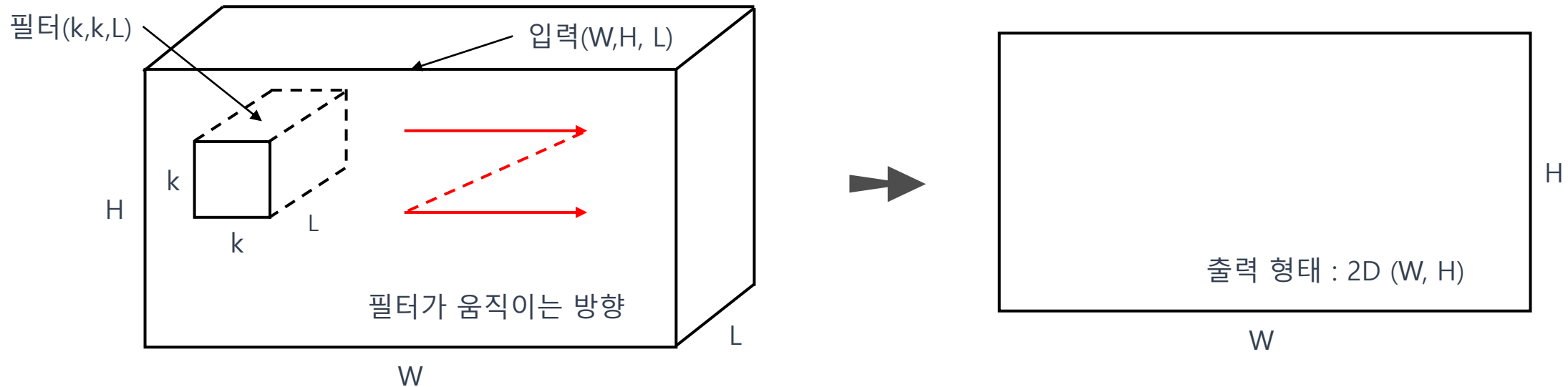
5.1.3 1D, 2D, 3D 합성곱

4) 3D 입력을 갖는 2D 합성곱

입력이 3D형태임에도, 출력 형태가 3D가 아닌 2D 행렬을 취하는 것
필터에 대한 길이(L)가 입력 채널의 길이(L)과 같아야 함.

입력(W,H,L)에 필터(k,k,L)를 적용하면 출력은 (W, H)

대표적 사례로는 **LeNet-5, VGG**



- 입력 : W 너비(Width), H 높이(Height), L 길이(Length)
- 필터 : k x k (높이x너비), L 길이(Length)
- 출력 : W 너비(Width), H 높이(Height), L 길이(Length)

5.1.3 1D, 2D, 3D 합성곱

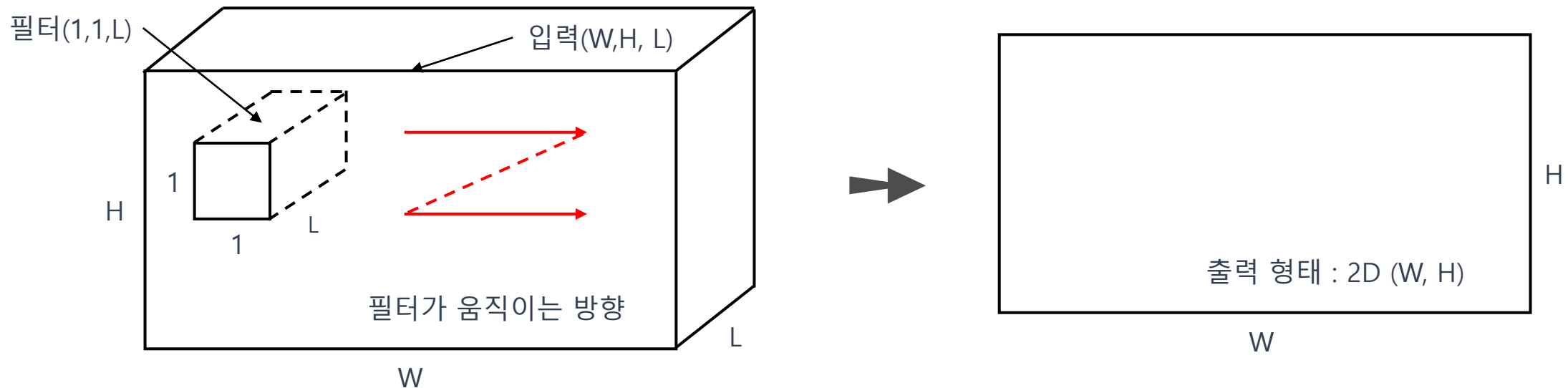
5) 1x1 합성곱

입력이 3D이고, 출력 형태가 2D

채널 수를 조정해서 연산량이 감소하는 효과

입력(W,H,L)에 필터(1,1,L)를 적용하면 출력은 (W, H)

대표적 사례로는 **GoogLeNet**



- 입력 : W 너비(Width), H 높이(Height), L 길이(Length)
- 필터 : 1 x 1 (높이x너비), L 길이(Length)
- 출력 : W 너비(Width), H 높이(Height), L 길이(Length)


```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader
```

```
#클래스 정의
classes = [
    "airplane",
    "bird",
    "car",
    "cat",
    "deer",
    "dog",
    "horse",
    "monkey",
    "ship",
    "truck"
]
```

```
# 1. 데이터 전처리
transform = transforms.ToTensor()
```

```
# 2. STL-10 데이터셋 로드
train_dataset = torchvision.datasets.STL10(
    root='./data',
    split='train',
    download=True,
    transform=transform
)
```

```
train_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
```

```
#3. CNN
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        # 합성곱층 1 + ReLU + 풀링 1
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(2)

        # 합성곱층 2 + 풀링 2
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

        # 완전연결층
        self.fc = nn.Linear(32 * 24 * 24, 10) # STL-10: 10 classes

    def forward(self, x):
        # Conv 1 → ReLU → Pool
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x) # 96 → 48

        # Conv 2 → ReLU → Pool
        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x) # 48 → 24

        # Flatten
        x = x.view(x.size(0), -1)

        # FC + Softmax
        x = self.fc(x)
        x = torch.softmax(x, dim=1)

    return x
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = CNN().to(device)

model.eval()

images, labels = next(iter(train_loader))
images = images.to(device)

output = model(images)

pred = torch.argmax(output, dim=1).item()
true = labels.item()

# 4. 데이터셋 시각화(참고용)
import matplotlib.pyplot as plt
img = images[0].cpu().permute(1, 2, 0) # (C,H,W) → (H,W,C)

plt.imshow(img)
plt.title(f"True: {classes[true]} | Pred: {classes[pred]}")
plt.axis("off")
plt.show()
```

+코드

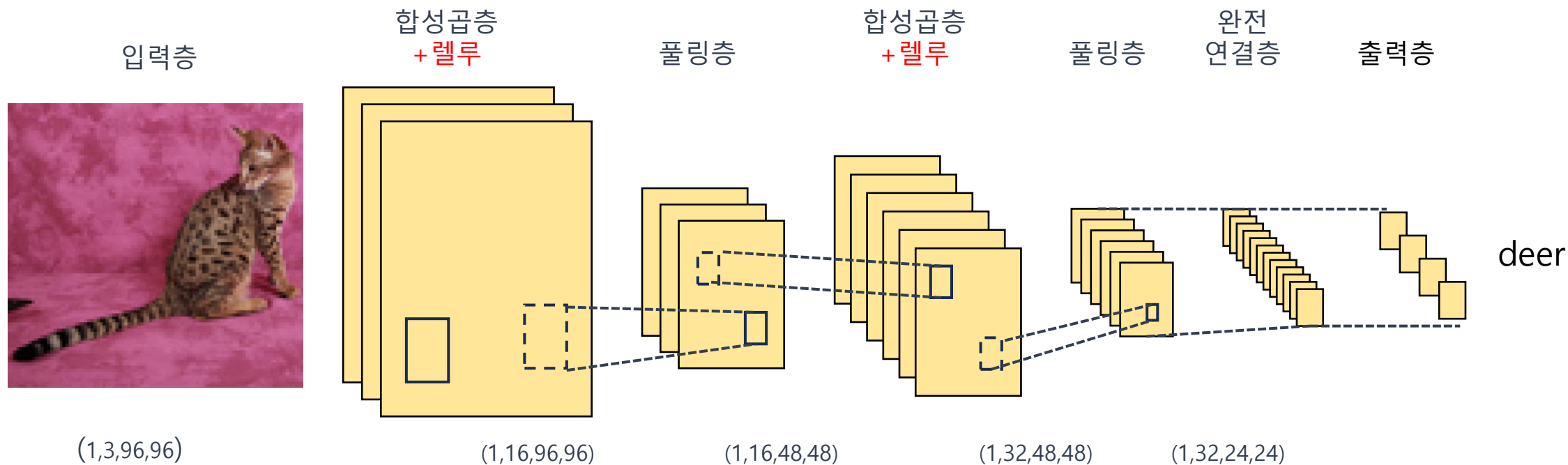
Layer (type:depth-idx)	Output Shape	Param #
CNN	[1, 10]	--
└─Conv2d: 1-1	[1, 16, 96, 96]	448
└─ReLU: 1-2	[1, 16, 96, 96]	--
└─MaxPool2d: 1-3	[1, 16, 48, 48]	--
└─Conv2d: 1-4	[1, 32, 48, 48]	4,640
└─ReLU: 1-5	[1, 32, 48, 48]	--
└─MaxPool2d: 1-6	[1, 32, 24, 24]	--
└─Linear: 1-7	[1, 10]	184,330

- 합성곱층 출력 데이터
- 풀링층 출력 데이터

$$W_2 = (W_1 - F + 2P)/S + 1 \quad W_2 = (W_1 - F)/S + 1$$

$$H_2 = (H_1 - F + 2P)/S + 1 \quad H_2 = (H_1 - F)/S + 1$$

$$D_2 = K \quad D_2 = D_1$$



True: cat | Pred: deer

감사합니다.

